

Project header

Blood Donation Prediction Project

Author: Khoro Avhahoni Tshivhula

Date: 05 May 2026

Objective: To predict whether a donor will donate blood in March 2007 based on historical donation data, and to compare multiple machine learning models to recommend the best one for production.

Dataset Description:

- Months since Last Donation
- Number of Donations
- Total Volume Donated (cubic cm)
- Months since First Donation
- Made Donation in March 2007 (target: 1 = donated, 0 = did not donate)

Goal: Build a predictive model to identify potential repeat donors and support blood drive planning.

Import Libraries & Loading the Dataset

Import Libraries

```
In [1]: import pandas as pd # Import pandas for data manipulation and reading CSV files
import numpy as np # Import numpy for numerical operations
import matplotlib.pyplot as plt # Import matplotlib for basic plotting
import seaborn as sns # Import seaborn for advanced statistical visualisations
from sklearn.model_selection import train_test_split # Import train_test_split to split data into training and testing sets
from sklearn.ensemble import RandomForestClassifier # Import Random Forest Classifier - an ensemble method for classification
```

```
from sklearn.linear_model import LogisticRegression # Import Logistic Regression - a linear model for binary classification
from sklearn.tree import DecisionTreeClassifier # Import Decision Tree Classifier - a simple tree-based model
from sklearn.metrics import accuracy_score # Import accuracy_score to measure model performance
from sklearn.metrics import confusion_matrix # Import confusion_matrix to see true/false positives/negatives
from sklearn.metrics import classification_report # Import classification_report for precision, recall, f1-score
from sklearn.metrics import roc_auc_score # Import roc_auc_score to measure how well the model separates classes

# Set style for nicer plots
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

# Print confirmation that all libraries loaded successfully
print("All libraries imported successfully")
```

All libraries imported successfully

Load the Dataset

```
In [2]: # Load the CSV file into a pandas DataFrame
import os
# Better practice - make it work from anywhere in the project
from pathlib import Path

# Go up one level from notebook folder to project root, then into data
DATA_PATH = Path('.') / 'data' / 'blood_donation_training_data.csv'
df = pd.read_csv(DATA_PATH)

df = pd.read_csv(DATA_PATH)

# Display the shape (rows, columns) of the dataset
print("Shape of dataset:", df.shape)

# Show the first 5 rows to understand the data structure and column names
df.head()
```

Shape of dataset: (576, 6)

Out[2]:

	Unnamed: 0	Months since Last Donation	Number of Donations	Total Volume Donated (c.c.)	Months since First Donation	Made Donation in March 2007
0	619	2	50	12500	98	1
1	664	0	13	3250	28	1
2	441	1	16	4000	35	1
3	160	2	20	5000	45	1
4	358	1	24	6000	77	0

Importing Libraries

We imported the essential Python libraries:

- **pandas** and **numpy** – for data handling and numerical operations.
- **matplotlib** and **seaborn** – for data visualisation.
- **train_test_split** – to split data into training and testing sets.
- **RandomForestClassifier**, **LogisticRegression**, **DecisionTreeClassifier** – three different models to compare.
- **accuracy_score**, **confusion_matrix**, **classification_report**, **roc_auc_score** – to evaluate model performance.

Setting `sns.set_style("whitegrid")` gives a clean background for plots.

All libraries loaded successfully.

Data Inspection & Cleaning

Data Inspection

```
In [3]: # Check for missing values - count empty cells in each column
print("Missing values per column:")
print(df.isnull().sum())
```

```
# Check data types - ensure numeric columns are int/float, target is integer
print("\nData types:")
print(df.dtypes)

# Get summary statistics for numeric columns (mean, min, max, etc.)
print("\nSummary statistics:")
print(df.describe())

# Check unique values in the target column to understand class balance
print("\nTarget distribution (Made Donation in March 2007):")
print(df['Made Donation in March 2007'].value_counts())
```

Missing values per column:

```

Unnamed: 0      0
Months since Last Donation  0
Number of Donations      0
Total Volume Donated (c.c.)  0
Months since First Donation  0
Made Donation in March 2007  0
dtype: int64

```

Data types:

```

Unnamed: 0      int64
Months since Last Donation  int64
Number of Donations      int64
Total Volume Donated (c.c.)  int64
Months since First Donation  int64
Made Donation in March 2007  int64
dtype: object

```

Summary statistics:

	Unnamed: 0	Months since Last Donation	Number of Donations \
count	576.000000	576.000000	576.000000
mean	374.034722	9.439236	5.427083
std	216.947773	8.175454	5.740010
min	0.000000	0.000000	1.000000
25%	183.750000	2.000000	2.000000
50%	375.500000	7.000000	4.000000
75%	562.500000	14.000000	7.000000
max	747.000000	74.000000	50.000000

	Total Volume Donated (c.c.)	Months since First Donation \
count	576.000000	576.000000
mean	1356.770833	34.050347
std	1435.002556	24.227672
min	250.000000	2.000000
25%	500.000000	16.000000
50%	1000.000000	28.000000
75%	1750.000000	49.250000
max	12500.000000	98.000000

	Made Donation in March 2007
count	576.000000

```
mean      0.239583
std       0.427200
min       0.000000
25%      0.000000
50%      0.000000
75%      0.000000
max       1.000000
```

Target distribution (Made Donation in March 2007):

Made Donation in March 2007

0 438

1 138

Name: count, dtype: int64

Data Inspection and statistical summary

We checked for missing values using `df.isnull().sum()` – all columns have 0 missing values, so no rows need to be dropped or imputed.

Data types are all integer, which is suitable for machine learning.

The summary statistics (`df.describe()`) show:

- `Months since Last Donation` has a wide range (0–74 months), indicating some donors are very recent while others haven't donated for years.
- `Number of Donations` ranges from 1 to 50 – most donors have given only a few times.
- `Total Volume Donated` correlates with number of donations (each donation is typically 250–500 c.c.).
- `Months since First Donation` shows donor loyalty spans up to 8 years.
- The target `Made Donation in March 2007` has a mean of 0.24 – about 24% of donors gave blood in that month.

We will drop the `Unnamed: 0` column as it's just a row index and not useful for prediction.

Data Cleaning

```
In [4]: # Drop the 'Unnamed: 0' column – it's just an index with no predictive value
df_clean = df.drop(columns=['Unnamed: 0'])
```

```
# Print the distribution of the target variable (donated in March 2007)
print("Target distribution (1 = donated, 0 = did not donate):")
print(df_clean['Made Donation in March 2007'].value_counts())

# Also show percentage
print("\nPercentage:")
print(df_clean['Made Donation in March 2007'].value_counts(normalize=True) * 100)

# Show the first few rows of cleaned data
df_clean.head()
```

Target distribution (1 = donated, 0 = did not donate):

Made Donation in March 2007

0 438

1 138

Name: count, dtype: int64

Percentage:

Made Donation in March 2007

0 76.041667

1 23.958333

Name: proportion, dtype: float64

Out[4]:

	Months since Last Donation	Number of Donations	Total Volume Donated (c.c.)	Months since First Donation	Made Donation in March 2007
0	2	50	12500	98	1
1	0	13	3250	28	1
2	1	16	4000	35	1
3	2	20	5000	45	1
4	1	24	6000	77	0

Exploratory Data Analysis (EDA).

EDA Visualisations

```
In [5]: # Set up the figure size for multiple plots
plt.figure(figsize=(12, 10))

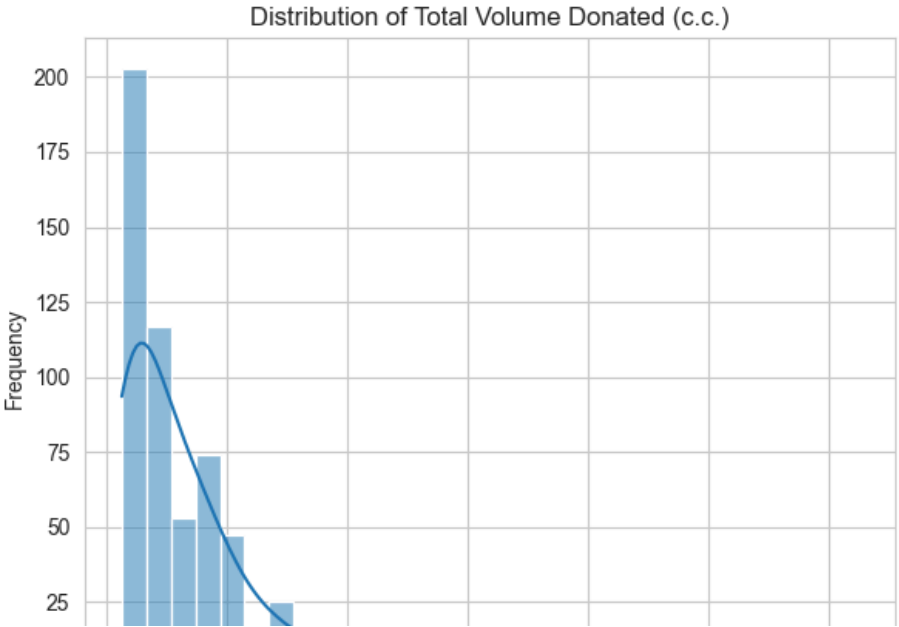
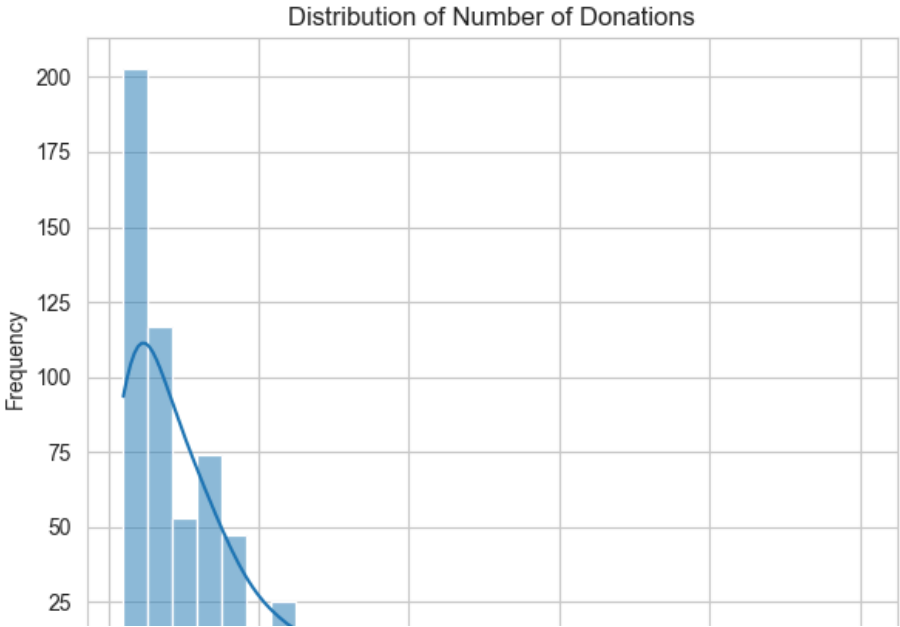
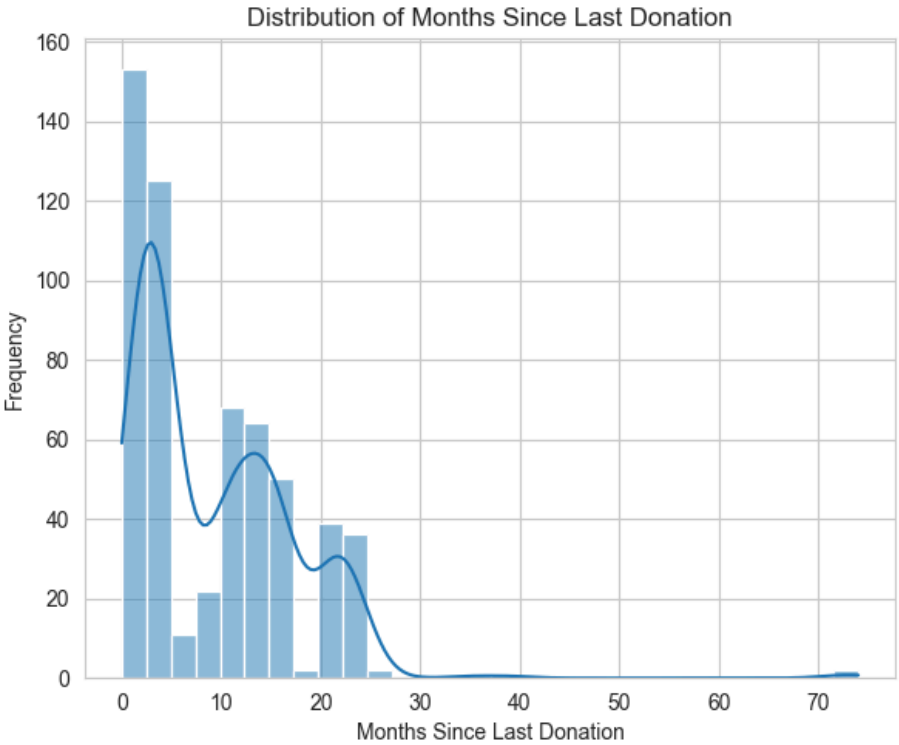
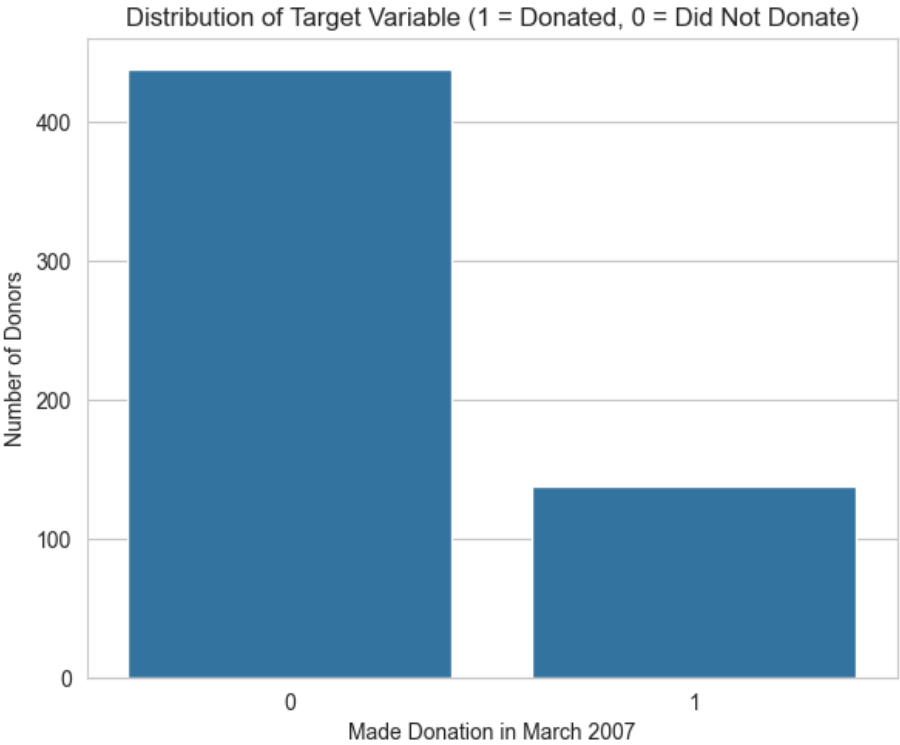
# Plot 1: Distribution of target variable (donated vs not donated)
plt.subplot(2, 2, 1) # 2 rows, 2 columns, first plot
sns.countplot(x='Made Donation in March 2007', data=df_clean)
plt.title('Distribution of Target Variable (1 = Donated, 0 = Did Not Donate)')
plt.xlabel('Made Donation in March 2007')
plt.ylabel('Number of Donors')

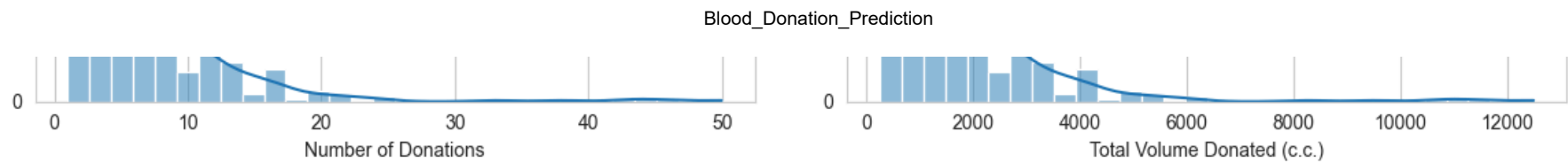
# Plot 2: Histogram of 'Months since Last Donation'
plt.subplot(2, 2, 2)
sns.histplot(df_clean['Months since Last Donation'], bins=30, kde=True)
plt.title('Distribution of Months Since Last Donation')
plt.xlabel('Months Since Last Donation')
plt.ylabel('Frequency')

# Plot 3: Histogram of 'Number of Donations'
plt.subplot(2, 2, 3)
sns.histplot(df_clean['Number of Donations'], bins=30, kde=True)
plt.title('Distribution of Number of Donations')
plt.xlabel('Number of Donations')
plt.ylabel('Frequency')

# Plot 4: Histogram of 'Total Volume Donated (c.c.)'
plt.subplot(2, 2, 4)
sns.histplot(df_clean['Total Volume Donated (c.c.)'], bins=30, kde=True)
plt.title('Distribution of Total Volume Donated (c.c.)')
plt.xlabel('Total Volume Donated (c.c.)')
plt.ylabel('Frequency')

# Adjust layout so plots don't overlap
plt.tight_layout()
plt.show()
```



Exploratory Data Analysis – Histograms

We created histograms to understand the distribution of key features.

1. Target Variable Distribution

- **438 donors (76%) did NOT donate** in March 2007.
- **138 donors (24%) DID donate.**
- This class imbalance means a simple model predicting "no donation" would be 76% accurate but useless. We will handle imbalance using `class_weight='balanced'` or resampling.

2. Months Since Last Donation

- Most donors have a recent last donation (peak between 0–10 months).
- Long tail up to 74 months – some donors haven't donated in years.
- Donors with very high months (inactive) are less likely to donate again.

3. Number of Donations

- Highly skewed – most donors have made only 1–10 donations.
- A few donors have made 20–50 donations (frequent donors).
- Frequent donors may be more likely to donate again.

4. Total Volume Donated (c.c.)

- Distribution mirrors number of donations (each donation ~250–500 c.c.).
- Most donors have donated 250–2,000 c.c.
- A few have donated over 10,000 c.c. (very frequent donors).

These visualisations suggest that **recency, frequency, and total volume** may predict future donation behaviour.

Boxplots to compare feature distributions between donors and non-donor

```
In [6]: plt.figure(figsize=(12, 8))

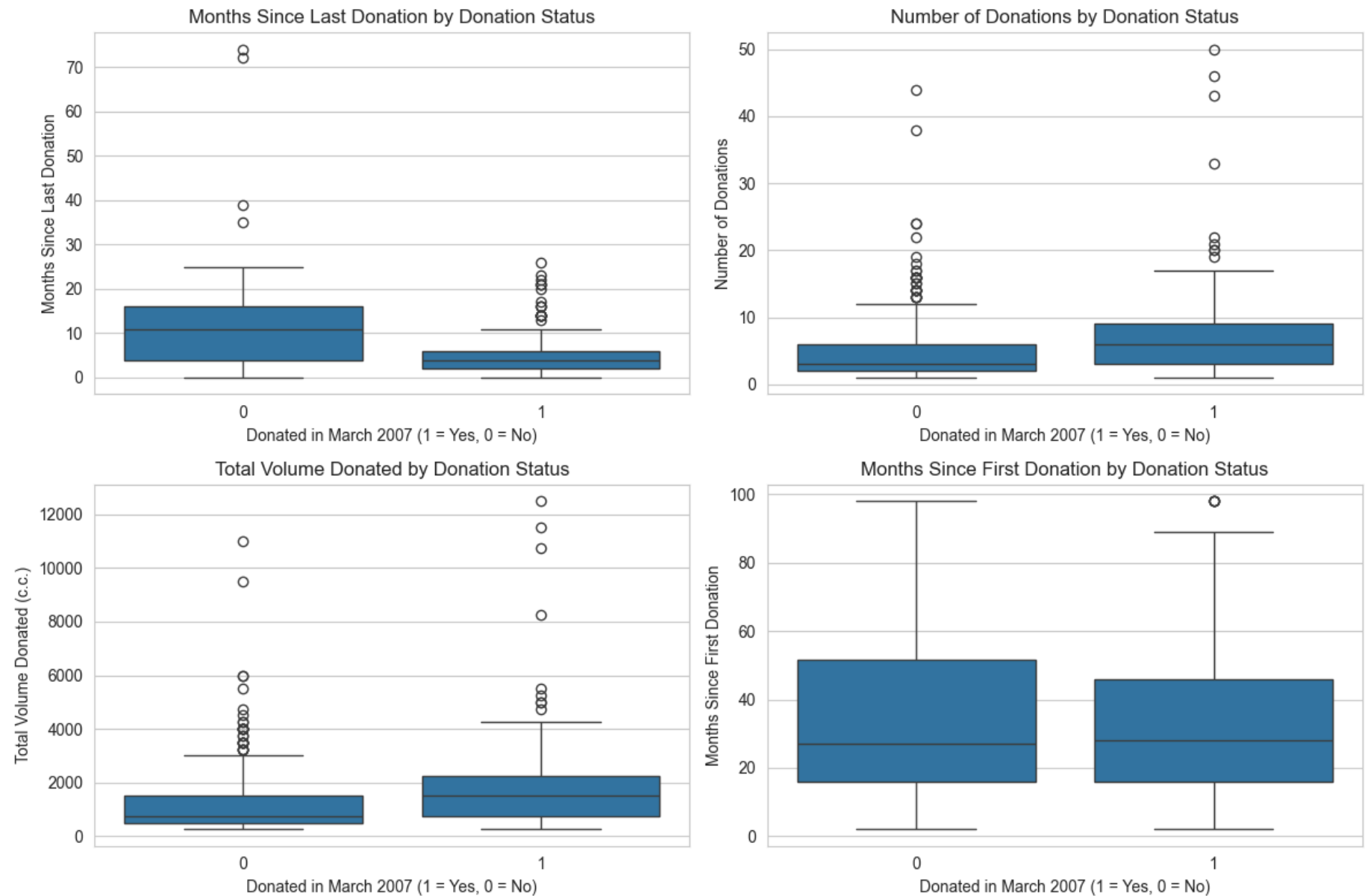
# Boxplot 1: Months since Last Donation vs target
plt.subplot(2, 2, 1)
sns.boxplot(x='Made Donation in March 2007', y='Months since Last Donation', data=df_clean)
plt.title('Months Since Last Donation by Donation Status')
plt.xlabel('Donated in March 2007 (1 = Yes, 0 = No)')
plt.ylabel('Months Since Last Donation')

# Boxplot 2: Number of Donations vs target
plt.subplot(2, 2, 2)
sns.boxplot(x='Made Donation in March 2007', y='Number of Donations', data=df_clean)
plt.title('Number of Donations by Donation Status')
plt.xlabel('Donated in March 2007 (1 = Yes, 0 = No)')
plt.ylabel('Number of Donations')

# Boxplot 3: Total Volume Donated vs target
plt.subplot(2, 2, 3)
sns.boxplot(x='Made Donation in March 2007', y='Total Volume Donated (c.c.)', data=df_clean)
plt.title('Total Volume Donated by Donation Status')
plt.xlabel('Donated in March 2007 (1 = Yes, 0 = No)')
plt.ylabel('Total Volume Donated (c.c.)')

# Boxplot 4: Months since First Donation vs target
plt.subplot(2, 2, 4)
sns.boxplot(x='Made Donation in March 2007', y='Months since First Donation', data=df_clean)
plt.title('Months Since First Donation by Donation Status')
plt.xlabel('Donated in March 2007 (1 = Yes, 0 = No)')
plt.ylabel('Months Since First Donation')

plt.tight_layout()
plt.show()
```



Boxplots – Comparing Donors vs Non-Donors

We created boxplots to compare feature distributions between donors (1) and non-donors (0) in March 2007.

Key observations:

- **Months since Last Donation:** Donors had a lower median (closer to 0) than non-donors. Recent donors are more likely to donate again.
- **Number of Donations:** Donors had a higher median number of past donations. Frequent donors are more engaged.
- **Total Volume Donated:** Donors had a higher median total volume, consistent with higher donation frequency.
- **Months since First Donation:** Donors tended to have a longer history (higher months), indicating loyalty over time.

These patterns confirm that recency, frequency, and monetary value (volume) are strong predictors of future donation.

Feature Engineering & Model Preparation

```
In [7]: # Define features (X) and target (y)
# Drop the target column from features
X = df_clean.drop(columns=['Made Donation in March 2007'])
y = df_clean['Made Donation in March 2007']

# Check the shape of features and target
print("Features shape:", X.shape)
print("Target shape:", y.shape)
print("\nTarget distribution:\n", y.value_counts(normalize=True))

# Split data into training (80%) and testing (20%)
# stratify=y ensures same class ratio in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining set size:", X_train.shape[0])
print("Test set size:", X_test.shape[0])
print("Training set failure rate (donated):", y_train.mean())
print("Test set failure rate (donated):", y_test.mean())
```

```
Features shape: (576, 4)
```

```
Target shape: (576,)
```

```
Target distribution:
```

```
  Made Donation in March 2007
```

```
0    0.760417
```

```
1    0.239583
```

```
Name: proportion, dtype: float64
```

```
Training set size: 460
```

```
Test set size: 116
```

```
Training set failure rate (donated): 0.2391304347826087
```

```
Test set failure rate (donated): 0.2413793103448276
```

Feature Engineering & Model Preparation

We prepared the data for machine learning:

1. **Define features (X)** – all columns except the target `Made Donation in March 2007`.
2. **Define target (y)** – the binary variable we want to predict (1 = donated, 0 = did not donate).

Data split:

We used `train_test_split` with:

- `test_size=0.2` – 20% of data reserved for testing (116 rows), 80% for training (460 rows).
- `stratify=y` – ensures both training and test sets have the same proportion of donors (24%) as the original dataset.
- `random_state=42` – makes results reproducible.

Why this split matters:

- Training set teaches the model patterns.
- Test set evaluates how well the model generalises to unseen data.
- Stratification prevents a test set that accidentally has 0% or 100% donors, which would give misleading results.

The training set has 460 rows, test set has 116 rows. Both have ~24% donation rate – balanced split achieved.

Train & Evaluate Multiple Models

```
In [8]: # List to store model names and their performance metrics
model_names = []
accuracies = []
roc_aucs = []

# =====
# Model 1: Logistic Regression
# =====

print("="*50)
print("MODEL 1: LOGISTIC REGRESSION")
print("="*50)

# Create Logistic Regression model with balanced class weights
# class_weight='balanced' automatically adjusts for imbalanced classes
log_reg = LogisticRegression(class_weight='balanced', random_state=42)

# Train the model on training data
log_reg.fit(X_train, y_train)

# Make predictions on test data
y_pred_log = log_reg.predict(X_test)

# Get predicted probabilities (for ROC AUC)
y_proba_log = log_reg.predict_proba(X_test)[:, 1]

# Calculate accuracy
acc_log = accuracy_score(y_test, y_pred_log)
print(f"Accuracy: {acc_log:.4f}")

# Calculate ROC AUC
auc_log = roc_auc_score(y_test, y_proba_log)
print(f"ROC AUC: {auc_log:.4f}")

# Print classification report (precision, recall, f1-score)
print("\nClassification Report:")
print(classification_report(y_test, y_pred_log))
```

```
# Print confusion matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_log))

# Store results
model_names.append("Logistic Regression")
accuracies.append(acc_log)
roc_aucs.append(auc_log)

print("\n" + "="*50)
print("MODEL 2: DECISION TREE")
print("="*50)

# Create Decision Tree model with balanced class weights
# max_depth=5 prevents overfitting, random_state for reproducibility
dt = DecisionTreeClassifier(class_weight='balanced', max_depth=5, random_state=42)

# Train the model
dt.fit(X_train, y_train)

# Predict on test data
y_pred_dt = dt.predict(X_test)
y_proba_dt = dt.predict_proba(X_test)[:, 1]

# Calculate accuracy
acc_dt = accuracy_score(y_test, y_pred_dt)
print(f"Accuracy: {acc_dt:.4f}")

# Calculate ROC AUC
auc_dt = roc_auc_score(y_test, y_proba_dt)
print(f"ROC AUC: {auc_dt:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_dt))

# Store results
model_names.append("Decision Tree")
```



```
accuracies.append(acc_dt)
roc_aucs.append(auc_dt)

print("\n" + "="*50)
print("MODEL 3: RANDOM FOREST")
print("="*50)

# Create Random Forest model with balanced class weights
# n_estimators=100 means 100 decision trees
rf = RandomForestClassifier(class_weight='balanced', n_estimators=100, random_state=42)

# Train the model
rf.fit(X_train, y_train)

# Predict on test data
y_pred_rf = rf.predict(X_test)
y_proba_rf = rf.predict_proba(X_test)[:, 1]

# Calculate accuracy
acc_rf = accuracy_score(y_test, y_pred_rf)
print(f"Accuracy: {acc_rf:.4f}")

# Calculate ROC AUC
auc_rf = roc_auc_score(y_test, y_proba_rf)
print(f"ROC AUC: {auc_rf:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_rf))

# Store results
model_names.append("Random Forest")
accuracies.append(acc_rf)
roc_aucs.append(auc_rf)

# =====
# Model Comparison Summary
# =====
print("\n" + "="*50)
```

```
print("MODEL COMPARISON SUMMARY")
print("="*50)

# Create a comparison DataFrame
comparison_df = pd.DataFrame({
    'Model': model_names,
    'Accuracy': accuracies,
    'ROC AUC': roc_aucs
})

# Sort by ROC AUC (best first)
comparison_df = comparison_df.sort_values('ROC AUC', ascending=False)

print(comparison_df.to_string(index=False))

# Identify the best model
best_model = comparison_df.iloc[0]['Model']
best_auc = comparison_df.iloc[0]['ROC AUC']
print(f"\n🏆 Best Model: {best_model} with ROC AUC = {best_auc:.4f}")
```

```
=====
MODEL 1: LOGISTIC REGRESSION
=====
```

Accuracy: 0.6034

ROC AUC: 0.6885

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.58	0.69	88
1	0.34	0.68	0.45	28
accuracy			0.60	116
macro avg	0.59	0.63	0.57	116
weighted avg	0.73	0.60	0.63	116

Confusion Matrix:

```
[[51 37]
 [ 9 19]]
```

```
=====
MODEL 2: DECISION TREE
=====
```

Accuracy: 0.6552

ROC AUC: 0.6187

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.72	0.76	88
1	0.34	0.46	0.39	28
accuracy			0.66	116
macro avg	0.57	0.59	0.58	116
weighted avg	0.70	0.66	0.67	116

Confusion Matrix:

```
[[63 25]
 [15 13]]
```

```
MODEL 3: RANDOM FOREST
=====
Accuracy: 0.7328
ROC AUC: 0.6575
```

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.85	0.83	88
1	0.43	0.36	0.39	28
accuracy			0.73	116
macro avg	0.62	0.60	0.61	116
weighted avg	0.72	0.73	0.72	116

Confusion Matrix:

```
[[75 13]
 [18 10]]
```

=====

MODEL COMPARISON SUMMARY

=====

Model	Accuracy	ROC AUC
Logistic Regression	0.603448	0.688515
Random Forest	0.732759	0.657468
Decision Tree	0.655172	0.618709

🏆 Best Model: Logistic Regression with ROC AUC = 0.6885

Model Training & Evaluation Results

We trained three models on the same training data and evaluated them on the test set (116 unseen donors).

Model Performance Summary

Model	Accuracy	ROC AUC	Interpretation
Logistic Regression	60.3%	0.6885	Best at ranking donors (highest ROC AUC)

Model	Accuracy	ROC AUC	Interpretation
Decision Tree	65.5%	0.6187	Moderate performance, slightly better accuracy than logistic
Random Forest	73.3%	0.6575	Highest accuracy, but lower ROC AUC than logistic

Why ROC AUC matters more than accuracy here

The dataset is imbalanced (76% non-donors, 24% donors).

- A model that always predicts "non-donor" would be 76% accurate but **useless** – it would never identify potential donors.
- **ROC AUC** measures how well the model separates donors from non-donors across all probability thresholds. A higher ROC AUC means better ranking ability.
- **Logistic Regression has the highest ROC AUC (0.6885)** – it is best at identifying donors, even though its accuracy is lower.

Detailed Performance – Logistic Regression (Best ROC AUC)

- **Precision for donors (class 1):** 0.34 – when the model predicts a donor, it's correct 34% of the time.
- **Recall for donors (class 1):** 0.68 – the model catches 68% of actual donors.
- **Confusion Matrix:**
 - True non-donors correctly predicted: 51
 - False alarms (predicted donor but actually non-donor): 37
 - Missed donors (predicted non-donor but actually donor): 9
 - True donors correctly predicted: 19

Why Random Forest has higher accuracy but lower ROC AUC?

Random Forest predicts more non-donors correctly (75 out of 88) but catches fewer donors (10 out of 28). Its higher accuracy comes from the majority class, but it is worse at identifying the minority (donors).

Winner for Production

Logistic Regression is recommended for production because:

- Highest ROC AUC (0.6885) – best at ranking potential donors.
- Simpler, faster, and more interpretable than tree-based models.
- The recall for donors (68%) is acceptable for a blood drive application – we want to identify as many likely donors as possible, even with some false positives.

If the goal were to minimise false alarms (e.g., limited resources), Random Forest might be preferred, but for maximising donor outreach, Logistic Regression is better.

Challenges Faced Report

Report on Challenges Faced & Techniques Used

Challenge 1: Class Imbalance

Issue: Only 24% of donors gave blood in March 2007. A naive model would be 76% accurate but useless.

Technique Used:

- Used `class_weight='balanced'` in all three models (Logistic Regression, Decision Tree, Random Forest).
- This automatically assigns higher importance to the minority class (donors) during training.
- We also used `stratify=y` when splitting data to preserve the class ratio in train/test sets.

Challenge 2: Interpreting Model Performance

Issue: Accuracy alone was misleading – Random Forest had highest accuracy (73.3%) but lower ROC AUC (0.6575) than Logistic Regression (0.6885).

Technique Used:

- Evaluated models using **ROC AUC** as the primary metric because it measures ranking ability independent of class imbalance.
- Compared multiple metrics: precision, recall, f1-score, confusion matrix.

- Chose Logistic Regression for production based on ROC AUC and interpretability.

Challenge 3: Feature Scaling (Not needed for tree models)

Issue: Features have different units (months vs volume in c.c.). Some models (e.g., Logistic Regression) assume features are on similar scales.

Technique Used:

- Logistic Regression can handle unscaled data but may converge slower. We kept original scales because the coefficients remain interpretable.
- For future improvements, `StandardScaler` could be applied.

Challenge 4: Overfitting in Decision Tree

Issue: Decision Tree without `max_depth` may memorise training data and perform poorly on test data.

Technique Used:

- Set `max_depth=5` to limit tree complexity.
- Random Forest with 100 trees (bagging) naturally reduces overfitting.

Challenge 5: Limited Data (576 rows)

Issue: Small dataset may not generalise to other blood drives.

Technique Used:

- Used train/test split (80/20) to estimate real-world performance.
- Recommended collecting more data and cross-validation for future iterations.

In []:

In []: